COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 Data Structures and Algorithms | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 07/29/2026 | **DATE SUBMITTED:** | 07/29/2026 |
| **NAME:** | Jiro Luis A. Katindig | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

**ACTIVITY NO. 4.1** **Array-Based vs. STL Stack Implementations**

## 1. PROCEDURE OUTPUT

**Tasks & Intended Learning Outcomes (ILOs):**

**ILO A:** Implement a stack using the C++ STL (std::stack).

**Whole code for ILO A**

```cpp
#include <iostream>
#include <stack>

int main(){

    std::stack<int> stack1;
    std::cout<<"----Testing the stack STL----" <<std::endl;

    std::cout<<"\n========== STACK 1 ==========" <<std::endl;

    //isEmpty
    std::cout<<"is the stack 1 empty? " << stack1.empty() << std::endl;

    //push
    stack1.push(10);
    std::cout << "The top of the stack 1 is: " << stack1.top() <<std::endl;
    stack1.push(9);
    std::cout << "The top of the stack 1 is: " << stack1.top() <<std::endl;
    stack1.push(8);
    std::cout << "The top of the stack 1 is: " << stack1.top() <<std::endl;
    stack1.push(7);
    std::cout << "The top of the stack 1 is: " << stack1.top() <<std::endl;

    stack1.pop();
    std::cout << "Popped out previous top. The top of the stack 1 is: " << stack1.top() <<std::endl;
    std::cout<<"is the stack 1 empty? " << stack1.empty() <<std::endl;
    std::cout << "The size of the stack 1 is: " << stack1.size() <<std::endl;


    std::stack<int> stack2;
    std::cout<<"\n========== STACK 2 ==========" <<std::endl;

    stack2.emplace(1);
    std::cout << "Emplaced 1. The top of the stack 2 is: " << stack2.top() <<std::endl;
    stack2.emplace(2);
    std::cout << "Emplaced 2. The top of the stack 2 is: " << stack2.top() <<std::endl;
    stack2.emplace(3);
    std::cout << "Emplaced 3. The top of the stack 2 is: " << stack2.top() <<std::endl;
    stack2.emplace(4);
    std::cout << "Emplaced 4. The top of the stack 2 is: " << stack2.top() <<std::endl;

    std::cout << "\nBefore swap:" <<std::endl;
    std::cout << "The top of the stack 1 is: " << stack1.top() <<std::endl;
    std::cout << "The top of the stack 2 is: " << stack2.top() <<std::endl;

    stack1.swap(stack2);
    std::cout << "\nAfter swap:" <<std::endl;
    std::cout << "The top of the stack 1 is: " << stack1.top() <<std::endl;
    std::cout << "The top of the stack 2 is: " << stack2.top() <<std::endl;

}
```

In this task, I created and tested two separate stacks to observe how data is stored, managed, and swapped. I started by checking that my first stack was empty, then added four numbers (10, 9, 8, and 7) one by one, confirming that the last number added always stayed on top. Next, I removed the top number (7), leaving 8 as the new top and three items remaining in the stack. I then created a second stack and added four numbers (1, 2, 3, and 4) to it. Finally, I checked the top values of both stacks before swapping their entire contents, successfully demonstrating that the two stacks completely traded all of their items with each other.

**Implementation of Emplace and Swap**

```
std::stack<int> stack2;
std::cout<<"\n========== STACK 2 ==========" <<std::endl;

stack2.emplace(1);
std::cout << "Emplaced 1. The top of the stack 2 is: " << stack2.top() <<std::endl;
stack2.emplace(2);
std::cout << "Emplaced 2. The top of the stack 2 is: " << stack2.top() <<std::endl;
stack2.emplace(3);
std::cout << "Emplaced 3. The top of the stack 2 is: " << stack2.top() <<std::endl;
stack2.emplace(4);
std::cout << "Emplaced 4. The top of the stack 2 is: " << stack2.top() <<std::endl;

std::cout << "\nBefore swap:" <<std::endl;
std::cout << "The top of the stack 1 is: " << stack1.top() <<std::endl;
std::cout << "The top of the stack 2 is: " << stack2.top() <<std::endl;

stack1.swap(stack2);
std::cout << "\nAfter swap:" <<std::endl;
std::cout << "The top of the stack 1 is: " << stack1.top() <<std::endl;
std::cout << "The top of the stack 2 is: " << stack2.top() <<std::endl;
```

In this part, I focused on populating a second stack and swapping its contents with the first stack. I added the numbers 1, 2, 3, and 4 into the second stack, verifying after each addition that the newest number sat on top. Next, I checked the top value of each stack before swapping them. After executing the swap, I checked the top elements once more to confirm that both stacks had successfully traded their contents.

**Output of ILO A**

```
----Testing the stack STL----

========== STACK 1 ==========
is the stack 1 empty? 1
The top of the stack 1 is: 10
The top of the stack 1 is: 9
The top of the stack 1 is: 8
The top of the stack 1 is: 7
Popped out previous top. The top of the stack 1 is: 8
is the stack 1 empty? 0
The size of the stack 1 is: 3
```

```
========== STACK 2 ==========
Emplaced 1. The top of the stack 2 is: 1
Emplaced 2. The top of the stack 2 is: 2
Emplaced 3. The top of the stack 2 is: 3
Emplaced 4. The top of the stack 2 is: 4

Before swap:
The top of the stack 1 is: 8
The top of the stack 2 is: 4

After swap:
The top of the stack 1 is: 4
The top of the stack 2 is: 8
```

This output shows the step-by-step results of my stack operations. First, it confirms that my first stack started off empty (shown as 1), then tracks the top value as I added numbers up to 7. After removing 7, it displays the new top value of 8, confirms the stack is no longer empty (shown as 0), and shows a total count of 3 items remaining. Next, it displays the top value of my second stack updating as I added numbers 1 through 4. Finally, it compares the top values before and after the swap, demonstrating that my first stack's top changed from 8 to 4 while my second stack's top changed from 4 to 8.

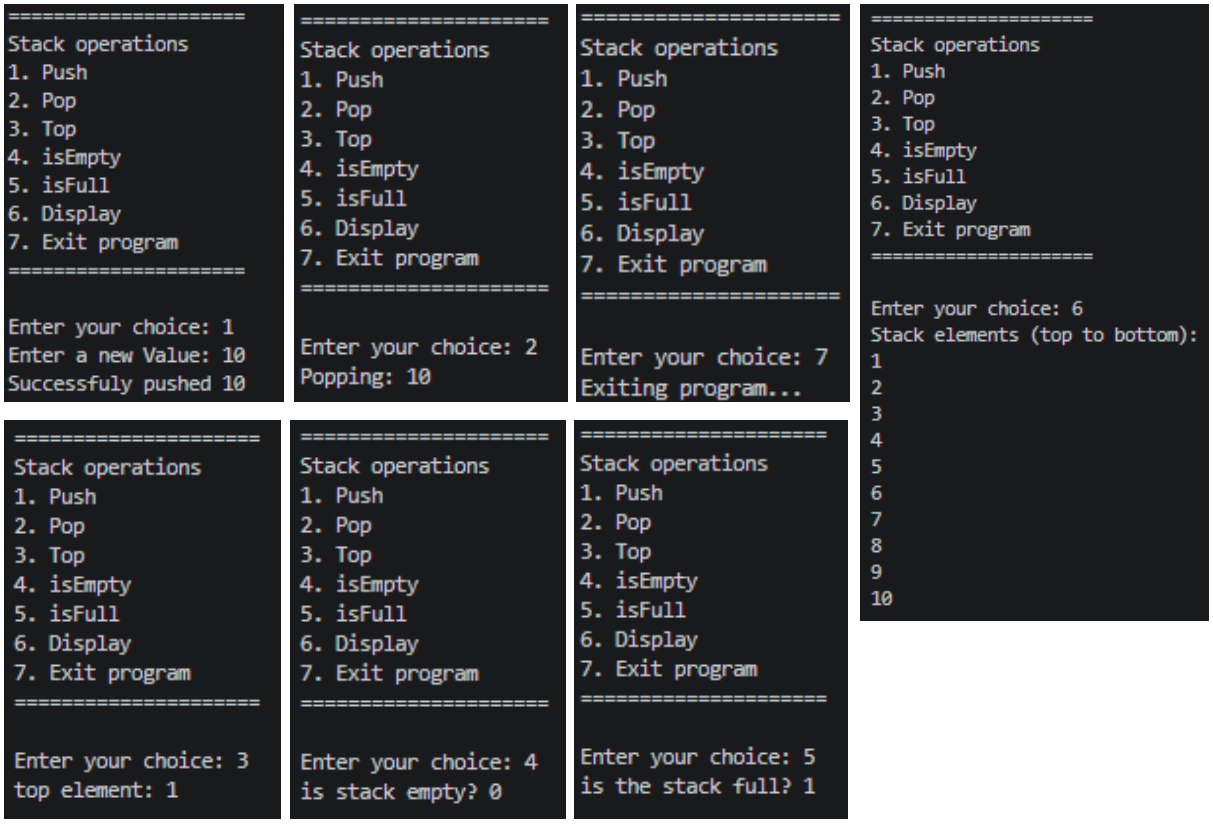
**ILO B.1:** Implement a stack using a Static Array.

**Whole code for ILO B**

```cpp
#include <iostream>

//global declarations
#define maxCap 10


int stackArr[maxCap];
int top = -1, newData;

// prototype functions
void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void display();

int main(){
// main driver
    int choice;
    while (true){
        std::cout<<"====================="<<std::endl;
        std::cout<<"Stack operations \n";
        std::cout<<"1. Push\n2. Pop\n3. Top\n4. isEmpty\n5. isFull\n6. Display\n7. Exit program"<<std::endl;
        std::cout<<"====================="<<std::endl;
        std::cout<<"\nEnter your choice: ";
        std::cin >>choice;

        switch(choice){
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
            case 3:
                Top();
                break;
            case 4:
                std::cout<<"is stack empty? "<< isEmpty()<<std::endl;
                break;
            case 5:
                std::cout<<"is the stack full? "<< isFull()<<std::endl;
                break;
            case 6:
                display();
                break;
            case 7:
                std::cout<<"Exiting program...";
                return 0;
            default:
                std::cout<<"Invalid choice." <<std::endl;
                break;

        }
    }
}

//function definition

bool isEmpty(){
    //how do we verify if the stack is empty
    if(top == -1) return true;
    return false;
}

bool isFull(){
    //how do we verify if stack is full?
    if (top == maxCap -1) return true;
    return false;
}

void push(){
    //error checking ??
    if(isFull()){
        std::cout <<"Stack overflow" << std::endl;
        return;
    }
    //pushing to the stack
    std::cout<<"Enter a new Value: ";
    std::cin >> newData;
    std::cout << std::endl;

    // how do we insert the data into the stack?
    stackArr[++top] = newData;
}

void pop(){
    //error checking
    if(isEmpty()){
        std::cout<<"Stack underflow" <<std::endl;
    }

    //Display the value that we are going to pop:
    std::cout<<"Popping: "<<stackArr[top]<<std::endl;
    //Decrement the top value from the stack
    top--;
}

void Top(){
    //error catching:
    if(isEmpty()){
        std::cout<<"The stack is empty" <<std::endl;
        return;
    }
    //check the top value
    std::cout <<"top element: "<<stackArr[top]<<std::endl;
}

void display() {
    if (isEmpty()) {
        std::cout << "The stack is empty." << std::endl;
        return;
    }

    std::cout << "Stack elements (top to bottom):" << std::endl;
    for (int i = top; i >= 0; i--) {
        std::cout << stackArr[i] << std::endl;
    }
}
```

In this task, I built a stack data structure from scratch using a static array with max cap. I implemented all the core stack operations, including adding new elements, removing the top element, viewing the top value, and checking whether the stack is empty or full. To make the program interactive, I created a user menu loop that allows choosing any of these operations in real time, displaying the stack's contents from top to bottom, and cleanly exiting the program when finished.

**Output of ILO B**




This output demonstrates the practical execution of my menu-driven stack program. I tested pushing a value (10) onto the stack and successfully popping it out. I verified the status checks by confirming the stack was not empty (0), checking the current top element (1), and confirming when the stack reached full capacity (1). I also displayed all stored elements vertically from top to bottom (1 through 10), and lastly tested the exit option to cleanly terminate the program.

**Implementation of Display**

```
void display() {
    if (isEmpty()) {
        std::cout << "The stack is empty." << std::endl;
        return;
    }

    std::cout << "Stack elements (top to bottom):" << std::endl;
    for (int i = top; i >= 0; i--) {
        std::cout << stackArr[i] << std::endl;
    }
}
```

In this part, I created the function responsible for displaying all the elements currently stored in the stack. I first included a check to verify if the stack was empty so it would notify the user if there were no items to show. If the stack contains data, I set it up to traverse backwards from the top index down to the very first item at the bottom, printing each value line by line.

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** _____

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** _____

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** _____

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.